

CENTINELA — Guía de estudio por integrante

Qué construyó cada uno, qué debe saber explicar y qué estudiar antes de la sustentación.

Sistema de detección de fraude en transacciones financieras · Azure · Sprints 1–6 · Documento generado el 29 de julio de 2026

Antes de usar este documento

El reparto se dedujo de las atribuciones que hay en el propio código y la documentación (comentarios del tipo «módulo Camila», «módulo Jorge»). Puede no reflejar exactamente quién hizo qué al final. **Revisad y corregid la asignación** antes de repartir el estudio: es más importante que cada uno domine lo que va a exponer que respetar esta deducción.

Parte común — esto lo debe saber todo el mundo

Cualquiera puede recibir una pregunta sobre el conjunto, no solo sobre su módulo. Estos cuatro puntos son obligatorios para todos.

1. Qué hace el sistema, en una frase

Recibe transacciones financieras, las puntúa con reglas de fraude y, cuando superan un umbral, abre un caso que un analista revisa en una consola web.

2. El flujo completo

Paso	Qué ocurre	Dónde
1	<code>POST /transactions</code> valida y responde <code>202</code> sin esperar	API de ingesta
2	Publica un evento en la cola <code>transactions</code>	Azure Queue Storage
3	El motor puntúa contra 4 reglas usando el historial	Worker + Cosmos DB
4	Si supera el umbral, publica el caso en la cola <code>cases</code>	Azure Queue Storage
5	Se persiste el caso con su explicación	PostgreSQL
6	El analista lo ve y lo gestiona	Consola Next.js

La pregunta que más probablemente os harán: ¿por qué colas?

Porque **desacoplan**. La API responde en milisegundos sin esperar al scoring, y si el motor se cae los mensajes **esperan en la cola** en vez de perderse. Cuando vuelve, los procesa. Cero pérdida de casos de fraude.

3. Arquitectura limpia (Clean Architecture)

El código está en cuatro capas y **las dependencias apuntan siempre hacia adentro**:

Capa	Qué contiene	De qué depende
<code>domain</code>	Entidades y reglas de negocio	De nada

application	Casos de uso	Solo de domain
infrastructure	Adaptadores: Azure, PostgreSQL...	De las interfaces de domain
presentation	API HTTP y worker	Compone todo lo demás

Por qué importa: el dominio no sabe que existe Azure. Por eso se puede probar todo el sistema sin desplegar nada — hay adaptadores en memoria que sustituyen a Cosmos, PostgreSQL y las colas. Las 159 pruebas corren sin tocar la nube.

4. Seguridad: cero credenciales

Regla transversal del proyecto. **Ningún secreto vive en el código, el repositorio, el pipeline ni las imágenes de contenedor.** Todo está en Azure Key Vault y se resuelve con *Managed Identity*: Azure le da a cada servicio una identidad propia y los permisos justos, sin contraseñas que alguien pueda filtrar.

Andrés — Motor de scoring y operabilidad

Reglas de fraude · Sprint 6 completo: contenedores, OCR, observabilidad, CI/CD

Qué construyó

El **motor que decide si una transacción es sospechosa**, y todo el Sprint 6, que llevó el sistema de App Service a contenedores y le añadió verificación documental y observabilidad.

Conceptos que debe dominar

Las cuatro reglas de fraude

Regla	Qué detecta	Puntos
Velocidad	Muchas transacciones en poco tiempo	25
Monto atípico	Importe muy superior al histórico de la cuenta	30
Geo imposible	Dos compras a distancia imposible de recorrer	45
Comercio de riesgo	Categoría marcada como peligrosa	20

Se **suman** los puntos; si el total llega al umbral (50 por defecto) se abre caso. El umbral es una variable de entorno: cambiarlo **no exige tocar código**.

Contenedores e imágenes multi-stage

Un contenedor empaqueta la aplicación con todo lo que necesita para correr. *Multi-stage* significa construir en una etapa y copiar a la imagen final solo el resultado, sin las herramientas de compilación.

Imagen	Tamaño	Reducción
Contenedorización ingenua	1240 MiB	—
API	174 MiB	–86 %
Worker	163 MiB	–87 %

De dónde salen los ahorros: imagen base *slim* (–1000 MiB), un juego de dependencias por carga (el worker no sirve HTTP, así que no lleva FastAPI), y no copiar herramientas de compilación al resultado final.

Escalado a cero con KEDA

KEDA vigila la longitud de la cola y crea réplicas del worker cuando hay mensajes. Sin trabajo **baja a cero réplicas y no cuesta nada**. El precio es el arranque en frío: el primer mensaje tarda ~60 s en procesarse.

Verificación documental (OCR)

El cliente adjunta el comprobante; Azure Document Intelligence extrae importe, fecha y comercio, y el sistema los **contrasta con la transacción**. Cuatro veredictos: `coincide`, `discrepa`, `insuficiente`, `ilegible`.

Decisión que hay que saber defender

El comercio no decide el veredicto, solo el importe y la fecha. La transacción guarda un identificador opaco (`m1`), no un nombre comercial. Si el comercio contara, un ticket real de «Supermercado Los Andes» nunca coincidiría con `m1` y el sistema marcaría como fraude cualquier comprobante legítimo.

Trazado distribuido

El problema: la API abría una traza y el worker otra, sin relación. No se podía responder «¿por qué este caso tardó 4 minutos?». La solución: el identificador de traza **viaja dentro del mensaje de la cola**, así que un solo `trace_id` recorre API → cola → worker → base de datos.

Documentos que debe estudiar

- `docs/fraud_scoring.md` — el motor y sus reglas
- `docs/container-optimization.md` — el reporte de optimización, con la metodología de medición
- `docs/document-verification.md` — OCR y veredictos
- `docs/observability.md` — trazado distribuido y métricas
- `DEPLOY.md` — cómo se levanta todo el sistema

Preguntas probables

— ¿Por qué contenedores y no App Service?

Portabilidad, escalado a cero (coste cero sin uso) y reproducibilidad: la imagen es idéntica en local y en producción.

— ¿Cómo se sabe que no hay secretos en las imágenes?

Hay un auditor que revisa **todas las capas**, no solo el resultado final — un secreto borrado en una capa posterior sigue siendo recuperable. Además tiene un autotest que construye una imagen con un secreto filtrado y comprueba que la auditoría la rechaza.

— ¿Qué pasa si el OCR no lee el documento?

Veredicto `ilegible`, que es un **resultado**, no un error: no se reintenta, porque daría lo mismo y gastaría cuota gratuita. Distinto de que el servicio esté caído, que sí se reintenta.

Camila — Mensajería y almacén de transacciones

Colas, desacoplamiento y persistencia del historial en Cosmos DB

Qué construyó

La **mensajería que desacopla** la API del motor de scoring, y el historial de transacciones en Cosmos DB que el motor consulta para decidir.

Conceptos que debe dominar

Por qué colas y no llamadas directas

Si la API llamara al motor directamente, tendría que **esperar** a que terminara, y si el motor estuviera caído la transacción **se perdería**. Con cola, la API responde `202 Accepted` en milisegundos y el mensaje espera hasta que alguien lo procese.

Garantía de entrega «al menos una vez»

Azure Queue Storage garantiza que un mensaje se entrega **al menos** una vez — puede entregarse dos. Por eso el sistema debe ser **idempotente**: procesar el mismo mensaje dos veces no puede duplicar un caso. Se resuelve con `ON CONFLICT DO UPDATE` en la base de datos.

Cómo funciona el borrado de mensajes (clave para explicar la resiliencia)

Al leer un mensaje **no se borra**: queda oculto un tiempo (*visibility timeout*). Solo se borra si el procesamiento fue bien. Si falla, reaparece y se reintenta. Tras varios intentos fallidos va a una **cola «poison»**, para que un mensaje corrupto no bloquee la cola para siempre.

Cosmos DB y la clave de partición

Cosmos es una base NoSQL. La **clave de partición** determina cómo se reparten los datos. Aquí es `/accountId`, y no es casualidad: el motor siempre consulta «las transacciones de esta cuenta», así que la consulta toca **una sola partición** — rápida y barata.

Ojo: la clave de partición **no se puede cambiar** tras crear el contenedor. Es una decisión de diseño irreversible.

Documentos que debe estudiar

- `docs/messaging.md` — colas y desacoplamiento
- `docs/nosql.md` — Cosmos DB y particionado
- `docs/architecture.md` — encaje general

Preguntas probables

— ¿Qué pasa si el motor de scoring se cae?

Nada se pierde. Los mensajes se acumulan en la cola y se procesan cuando vuelve. Es la diferencia entre una arquitectura acoplada y una desacoplada.

— ¿Por qué NoSQL y no SQL para las transacciones?

Volumen alto, escritura constante y consultas siempre por cuenta. No hacen falta relaciones complejas. Los **casos** sí van en PostgreSQL, porque ahí sí hay relaciones y auditoría.

Jorge — Infraestructura y worker

Despliegue automatizado, motor como servicio en segundo plano, integración

Qué construyó

Los **scripts que levantan todo el sistema en Azure** y el worker que procesa las colas en segundo plano.

Conceptos que debe dominar

Infraestructura como código

Toda la infraestructura se crea con scripts, **ningún clic en el portal**. Ventajas: es reproducible, está versionada, y cualquiera puede levantar el sistema entero. Los scripts son **idempotentes**: ejecutarlos varias veces no rompe nada, porque comprueban si el recurso ya existe.

Lección real del proyecto

Un fallo costó tiempo por confundir «el recurso **existe**» con «el recurso **sirve**». Un intento fallido dejó un entorno a medio crear; el reintento lo dio por bueno y se quedó esperando algo que nunca iba a funcionar. Ahora se comprueba también el *estado*, no solo la existencia.

El worker: de Azure Functions a contenedor

El motor corría como Azure Function (se activa sola al llegar un mensaje). Al contenedorizar se reescribió como proceso autónomo, porque la imagen base de Functions pesa más de 1 GB. Hubo que **reponer a mano** lo que el host daba gratis: reintento, cola *poison* y apagado limpio al recibir la señal de parada.

Managed Identity

En vez de guardar contraseñas, Azure le da a cada servicio una **identidad** y se le conceden permisos sobre los recursos que necesita. Sin contraseñas no hay nada que filtrar ni que rotar.

Documentos que debe estudiar

- `DEPLOY.md` — el paso a paso completo, con los fallos reales encontrados
- `docs/deployment-guide.md` — guía de despliegue
- `infra/deploy-all.sh` y `infra/containerapps.sh` — leer los comentarios, explican el porqué

Preguntas probables

— ¿Cuánto se tarda en levantar el sistema desde cero?

40–60 minutos, casi todo esperando a Azure. PostgreSQL y el entorno de contenedores son lo más lento.

— ¿Qué pasa si un despliegue falla a mitad?

Se vuelve a lanzar: los scripts son idempotentes y retoman donde estaban. Pero hay que borrar recursos que quedaron a medio crear, porque «existe» no es lo mismo que «funciona».

Juan José — Gestión de casos y consola

Almacén relacional, auditoría inmutable, interfaz del analista, red

Qué construyó

La **base de datos de casos** con su auditoría, y la **consola web** donde el analista los revisa, asigna y resuelve.

Conceptos que debe dominar

Por qué PostgreSQL para los casos

Los casos tienen **relaciones** (caso → asignaciones → resoluciones → auditoría) y exigen **consistencia transaccional**: crear un caso y su explicación debe ser atómico. Eso es exactamente para lo que sirve una base relacional. Las transacciones, en cambio, van a NoSQL por volumen.

Auditoría inmutable

La tabla de auditoría tiene un *trigger* que **impide modificar o borrar filas**: solo se puede insertar. En un sistema antifraude esto es imprescindible — si alguien pudiera alterar el registro, no valdría como prueba.

Explicaciones append-only

Las explicaciones tampoco se sobrescriben: se **añade** una versión nueva y la lectura toma la más reciente. Así queda traza de lo que se dijo primero. Esto es lo que permitió añadir el enriquecimiento asíncrono sin tocar la lectura.

El patrón BFF de la consola

Backend for Frontend: el navegador **nunca habla directamente con la API**. Habla con el servidor de Next.js, que añade la clave de analista y llama a la API. Así **la clave nunca llega al navegador**, donde cualquiera podría verla con las herramientas de desarrollo.

Esto se verificó en el despliegue real

Se comprobó que ni la clave ni su nombre aparecen en el HTML servido. Fue también la razón de desplegar la consola como contenedor y no como sitio estático: un sitio estático no tiene servidor donde esconder la clave.

Documentos que debe estudiar

- `docs/cases_store_spec.md` — modelo de datos de casos
- `docs/explainability.md` — cómo se explica una decisión
- `docs/network.md` y `README_NETWORKING.md` — red
- `scripts/init-cases-db.sql` — el esquema, con sus triggers

Preguntas probables

— ¿Por qué dos bases de datos distintas?

Cada una para lo que hace bien: NoSQL para volumen alto de transacciones con consultas simples por cuenta; relacional para casos, que tienen relaciones y exigen auditoría y consistencia.

– ¿Cómo se garantiza que nadie manipule la auditoría?

Un trigger a nivel de base de datos bloquea `UPDATE` y `DELETE` . Ni siquiera la aplicación puede alterarla: la protección está en la base, no en el código.

Lukas — Seguridad

Autenticación, autorización, Key Vault, rate limiting

Qué construyó

Todo lo que impide que cualquiera use la API, y el sistema que mantiene los secretos fuera del código.

Conceptos que debe dominar

Autenticación por API key con roles

Cada petición lleva una cabecera `X-API-Key`. Hay dos roles: **servicio** (puede enviar transacciones) y **administrador** (además puede listar y gestionar casos). Las claves **no están en el código**: viven en Key Vault.

Azure Key Vault y Managed Identity

Key Vault es la caja fuerte. La aplicación no lleva la contraseña de Key Vault — eso solo movería el problema. Usa **Managed Identity**: Azure la reconoce por su identidad y le entrega los secretos. **No hay ninguna contraseña en ninguna parte.**

Rate limiting

Limita cuántas peticiones acepta por IP en una ventana de tiempo (10/minuto por defecto). Protege de abuso y de ataques de denegación de servicio.

Un fallo real que enseña por qué importa el detalle

Al contenedorizar, la plataforma sondea `/health` cada pocos segundos desde la misma IP. Con el límite por defecto, esas sondas lo agotaban, recibían `429` y la plataforma **reiniciaba en bucle un contenedor sano**. Se exceptuaron las rutas de sonda — el resto sigue protegido, y hay una prueba que lo verifica.

Defensa en profundidad en las imágenes

Los contenedores corren como **usuario sin privilegios** (no root) y se les **quita el gestor de paquetes**: si alguien lograra ejecutar código dentro, no podría instalar herramientas ni tendría permisos de administrador.

Documentos que debe estudiar

- `docs/security.md` — modelo de seguridad completo
- `docs/api_contract.md` — contrato y autenticación
- `infra/security.sh` — cómo se cablean los secretos
- `infra/verify-image-secrets.sh` — la auditoría de imágenes

Preguntas probables

— ¿Dónde están las contraseñas?

En Azure Key Vault. La aplicación accede con Managed Identity, así que no necesita ninguna contraseña para obtenerlas. Es la respuesta al problema clásico de «¿y quién guarda la llave de la caja fuerte?».

— ¿Cómo saben que no se ha filtrado ningún secreto?

Hay una auditoría automática en el pipeline que revisa todas las capas de cada imagen y bloquea la publicación si encuentra algo. Y se verifica a sí misma: construye una imagen con un secreto filtrado y comprueba que la detecta.

Santiago (Chanti) — Datos y costos

Diseño del almacén NoSQL, control de presupuesto, elección de región

Qué construyó

El diseño del almacén de transacciones en Cosmos DB y el **control de costos** del proyecto.

Conceptos que debe dominar

Capacidad en Cosmos DB (RU/s)

Cosmos no cobra por consultas sino por **Request Units**: una unidad de trabajo. Se aprovisionan 400 RU/s, que caben enteras en la capa gratuita (1000 RU/s). Una consulta mal diseñada —que recorra varias particiones— consume muchas más RU y **cuesta más dinero**. Por eso la clave de partición importa.

TTL: borrado automático

Los documentos caducan a los 90 días. Es deliberado: la regla de fraude con ventana más amplia mira 90 días atrás, así que guardar más sería pagar por datos que nadie consulta.

Capas gratuitas usadas

Servicio	Capa gratuita
Cosmos DB	1000 RU/s + 25 GB
Container Apps	180 000 vCPU-s/mes (~200 h de una réplica)
Document Intelligence	F0: 500 páginas/mes
Log Analytics	5 GB/mes
GHCR (imágenes)	Gratis con paquetes públicos

Dato honesto que conviene tener a mano

El único recurso que cuesta de verdad es **PostgreSQL**, que está encendido 24/7. Los contenedores cuestan prácticamente cero gracias al escalado a cero. Se puede parar la base hasta 7 días para contener gasto, pero el sistema deja de funcionar mientras tanto.

Elección de región

No es arbitraria: influye en latencia, coste y **disponibilidad de servicios**. En este proyecto fue un problema real — PostgreSQL no se podía crear en varias regiones por restricciones de la suscripción, y una región llegó a rechazar la creación del entorno de contenedores **por falta de capacidad**.

Documentos que debe estudiar

- `docs/nosql.md` — diseño de Cosmos
- `docs/credit-report-nosql.md` — consumo y costos
- `docs/region_justification.md` — elección de región
- `docs/container-optimization.md` §5 — costos del despliegue en contenedores

Preguntas probables

– ¿Cuánto cuesta mantener esto?

Casi todo está en capas gratuitas. El gasto real es PostgreSQL. Los contenedores escalan a cero: sin uso, no cuestan.

– ¿Qué pasa si el volumen crece mucho?

Cosmos y Container Apps escalan solos; habría que subir las RU/s aprovisionadas y el máximo de réplicas. El diseño ya lo contempla: son variables de configuración.

Samuel — Almacenamiento de documentos

Blob Storage, subida de comprobantes, acceso temporal seguro

Qué construyó

El almacenamiento de los **comprobantes** que se adjuntan a un caso y el mecanismo para consultarlos de forma segura.

Conceptos que debe dominar

Por qué Blob Storage y no la base de datos

Los ficheros binarios (PDF, imágenes) no van en una base de datos: la encarecen y la ralentizan. Van a Blob Storage, y en la base solo se guarda la **referencia**.

URLs temporales (SAS)

El contenedor de blobs es **privado**. Para que el analista vea un documento se genera una **URL firmada que caduca** (5 minutos). Si esa URL se filtrara, en minutos deja de servir. Es mucho mejor que hacer público el contenedor.

Validación de subida

Antes de aceptar un fichero se comprueba **tipo** (solo PDF, PNG, JPEG) y **tamaño** (máximo 5 MB). Sin esas validaciones, cualquiera podría subir un ejecutable o llenar el almacenamiento.

Encadenado con la verificación documental

Al subir un documento se publica un evento en la cola `documents`. El worker lo recoge, descarga el fichero, lo pasa por OCR y contrasta los datos con la transacción. El almacenamiento es **el punto de entrada** de esa cadena.

Documentos que debe estudiar

- `docs/storage.md` — almacenamiento de documentos
- `docs/document-verification.md` — qué ocurre después de subir
- `docs/security.md` — la parte de acceso temporal

Preguntas probables

— ¿Cómo se protege el acceso a los documentos?

El contenedor es privado y el acceso va por URL firmada con caducidad de 5 minutos. No hay ninguna URL permanente que pueda filtrarse.

— ¿Por qué una cola al subir un documento?

Porque el OCR es lento. Sin cola, quien sube el fichero esperaría a que se analizara. Con cola, la respuesta es inmediata y el análisis ocurre después.

Consejos para la sustentación

Lo que suele marcar diferencia

- **Explicar el porqué, no el qué.** «Usamos colas» dice poco; «usamos colas porque si el motor se cae los casos de fraude no se pierden» demuestra criterio.
- **Conocer las decisiones y sus renunciaciones.** Toda decisión tuvo un coste. Escalado a cero: barato, pero con arranque en frío. NoSQL: rápido y escalable, pero sin relaciones.
- **Poder enseñarlo funcionando.** El sistema está desplegado: se puede enviar una transacción y ver aparecer el caso en la consola.

Ser honestos con lo que no está terminado

Un profesor valora más que reconozcáis un límite a que os pille en una afirmación falsa. Lo que **no** está cerrado:

- El **enriquecedor de explicaciones** tiene toda la tubería construida y probada, pero falta decidir el servicio que redactará el texto.
- El **despliegue automático desde el pipeline** está implementado pero no activado: requiere una configuración de identidad que se decidió no hacer. El despliegue es manual, aunque totalmente automatizado en scripts.
- Las **trazas no se ven en el portal de Application Insights** porque el agente gestionado de Azure no las acepta. Se siguen consultando por `trace_id` en los registros.

Demostración en vivo sugerida (5 minutos)

1. Abrir la consola y mostrar la bandeja de casos.
2. Enviar una transacción sospechosa desde la terminal → responde `202` al instante.
3. Explicar mientras tanto qué está pasando: cola → worker → reglas → caso.
4. Refrescar la consola: aparece el caso con su explicación.
5. Abrir un caso con comprobante y mostrar el veredicto del OCR.

Aviso para la demo: si el sistema lleva rato sin uso, la primera petición tarda unos segundos porque los contenedores están a cero réplicas. **No es un fallo, es el ahorro de coste funcionando** — y explicarlo así suma.